

08_socialLens_analysis

July 9, 2026

```
[1]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from consts import *
import numpy as np
from itertools import combinations
from scipy.stats import linregress
import scipy.stats as stats
import json
from matplotlib.patches import Patch

bigfs = 11
smallfs = 9
tinyfs = 7
plt.rcParams.update({"font.size":smallfs})
plt.rcParams.update({"figure.figsize":(16/2.54, 9/2.54)})
sns.set_style("ticks")
sns.set_context("paper")
```

```
[2]: fitmode = "fitAllDots"
df_p = pd.read_csv("processed_data/
↳2026-07-07_data_processed_participant_withAllIssueWeights.csv")
df_diff = pd.read_csv("processed_data/
↳2026-07-07_data_processed_differences_withAllIssueWeights.csv")
```

```
[3]: def p_to_stars(p):
    if p < 0.001: return "***"
    if p < 0.01:  return "**"
    if p < 0.05:  return "*"
    return "ns"
```

1 Analyse Social Circle Variance and how it relates to other things

2 Effect of Treatment on social circle variance

```
[4]: # Change in stdSC over two waves

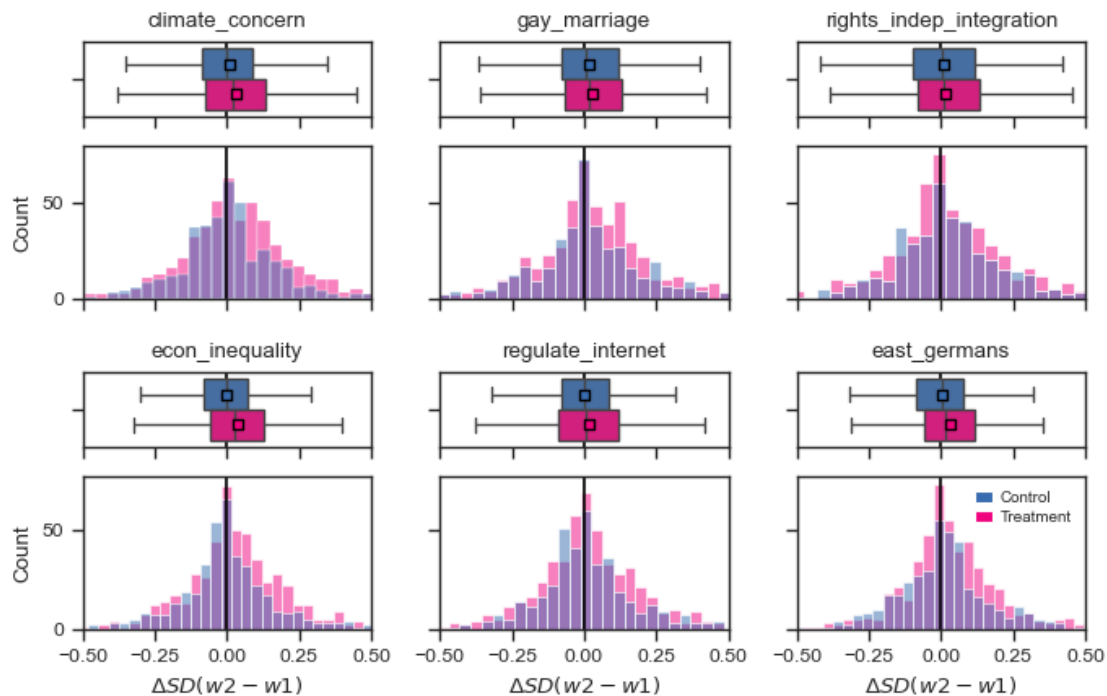
k = "std_socialCircle_ops"
kname = r"SD"
fig, axes = plt.subplots(5, 3, figsize=(7, 4.5), sharey="row", sharex=True,
    ↪height_ratios=[1,2,0.2,1,2])
for ax in axes[2,:].flatten():
    ax.axis("off")
for ax, ax2, q in zip(axes[[1,4], :].flatten(), axes[[0,3],:].flatten(),
    ↪questions_sc):
    ax.axvline(0,color="k")
    ax2.axvline(0,color="k")
    alpha_change = df_p.pivot_table(index="wave", columns="id",
    ↪values=f"{k}_{q}").diff(axis=0).query("wave==2").T.rename(columns={2:"diff"})
    alpha_change = alpha_change.join(df_p.query("wave==2")[["id",
    ↪"treatment_wave2"]].set_index("id"),)
    sns.histplot(alpha_change, x="diff", hue="treatment_wave2", ax=ax,
    ↪legend=False, palette=cmapTreatment)
    sns.boxplot(alpha_change, x="diff", hue="treatment_wave2", ax=ax2,
    ↪legend=False, fliersize=False, palette=cmapTreatment)
    mean_diff = alpha_change.groupby("treatment_wave2")["diff"].mean().
    ↪reset_index()
    sns.stripplot(x="diff", y=None, hue="treatment_wave2", data=mean_diff,
    ↪ax=ax2, legend=False, dodge=True, s=4, marker="s", linewidth=1,
    ↪edgecolor="k", palette=cmapTreatment)
    ax.set_xlabel(fr"$\Delta$ {kname}-{ '{(w2-w1)}' }$")
    ax2.set_title(f"{q}")
    ax.set_xlim(-0.5, 0.5)

legend_handles = [
    Patch(facecolor=cmapTreatment[0.], label="Control"),
    Patch(facecolor=cmapTreatment[1.], label="Treatment"),
]

axes[-1,-1].legend(
    handles=legend_handles,
    labels=["Control", "Treatment"],
    fontsize=7,
    frameon=False,
    loc="upper right",
    handlelength=1.2,
    handletextpad=0.4,
    labelspacing=0.3,
```

```
)
fig.tight_layout(h_pad=0)
plt.savefig(f"figs/{k}_Change_wave2.png", dpi=600)

# %%
```



[]:

2.0.1 Is opinion extremity relevant for reported social circle opinion variance?

Yeah, but very weakly.

```
[5]: ops = df_p.melt(id_vars=["id", "wave"], value_vars=[f"x_self_{q}" for q in
↳ questions_sc], value_name="opinion", var_name="question").
↳ replace(dict(zip([f"x_self_{q}" for q in questions_sc], questions_sc)))
stds = df_p.melt(id_vars=["id", "wave"],
↳ value_vars=[f"std_socialCircle_ops_{q}" for q in questions_sc],
↳ value_name="std opinions in social circle", var_name="question").
↳ replace(dict(zip([f"std_socialCircle_ops_{q}" for q in questions_sc],
↳ questions_sc)))
ops_std = ops.merge(stds, on=["id", "wave", "question"])
ops_std["|opinion|"] = abs(ops_std["opinion"])

g = sns.lmplot(
```

```

ops_std,
y="std opinions in social circle",
x="|opinion|",
hue="question",
height=3.5,
aspect=1.2,
x_jitter=0.02,
scatter_kws={"s": 1, "alpha": 0.15,},
line_kws={"linewidth": 1.3},
legend=False, # we'll draw a compact one manually
)

ax = g.axes[0, 0]
palette = sns.color_palette(n_colors=len(questions_sc))

slopes, stars_list = [], []
for question in questions_sc:
    subset = ops_std[ops_std["question"] == question].dropna(
        subset=["std opinions in social circle", "|opinion|"]
    )
    slope, _, _, p, _ = stats.linregress(
        subset["std opinions in social circle"],
        subset["|opinion|"]
    )
    slopes.append(slope)
    stars_list.append(p_to_stars(p))
qmap = dict(zip(questions_sc, ["climate concern", "gay marriage", "migrant_
↳rights", "inequality", "digital regulation", "east germans"]))
handles = [
    plt.Line2D([0], [0], color=palette[i], linewidth=2,
               label=f"{qmap[q]} = {slope:.2f}{stars}")
    for i, (q, slope, stars) in enumerate(zip(questions_sc, slopes, stars_list))
]

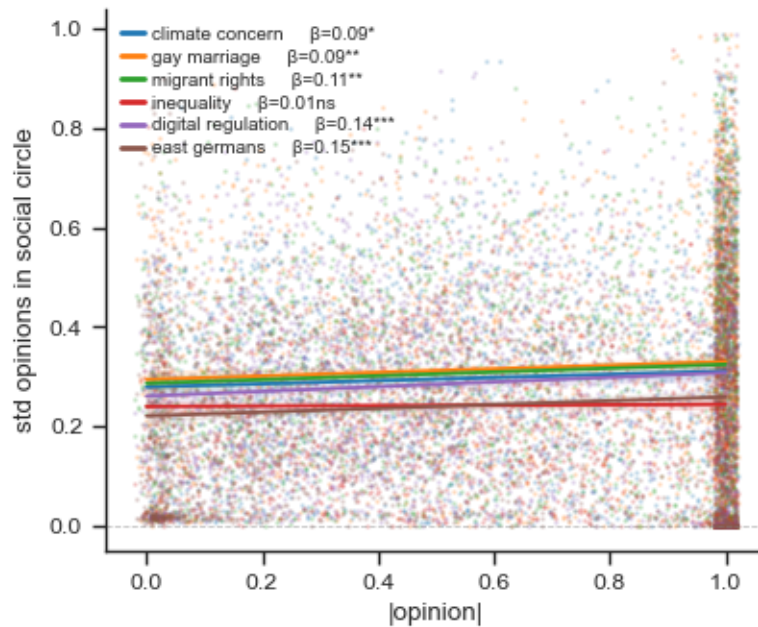
ax.legend(handles=handles, fontsize=7, frameon=False,
          loc="upper left", handlelength=1.2,
          handletextpad=0.4, labelspace=0.3)

# ax.set_title(f"Issue weight vs. social circle opinion spread", fontsize=9,
↳pad=8)
ax.axhline(0, color="gray", linewidth=0.6, linestyle="--", alpha=0.5) #
↳reference line at 0

g.figure.tight_layout()
g.figure.savefig("figs/corr_absOpinion_stdSC.png", dpi=150, bbox_inches="tight")

```

```
# sns.lmplot(ops_std, hue="question", x="abs_opinion", y="std",
↳ scatter_kws={"s":1, "alpha":0.3})
```



3 Are issue weights related to social circle variance?

```
[6]: # Linear Regression of Correlation over Social Circle Variance
waves = [1]
wavecondition=f"wave in {waves}"
k = "corrP"
kname = r"correlation $r_{d,\Delta X}$"
stdname = "opinion std in social circle"
corr_cols = [f"{k}_alpha_{fitmode}_{q}" for q in questions_sc]
stdSC_cols = [f"std_socialCircle_ops_{q}" for q in questions_sc]
corrs = df_p.query(wavecondition)[["id", "wave", "treatment_wave2"] + corr_cols].
↳ melt(id_vars=["id", "wave", "treatment_wave2"], value_name=k,
↳ var_name="question").replace(dict(zip(corr_cols, questions_sc)))
stds = df_p.query(wavecondition)[["id", "wave", "treatment_wave2"] + stdSC_cols].
↳ melt(id_vars=["id", "wave", "treatment_wave2"], value_name=stdname,
↳ var_name="question").replace(dict(zip(stdSC_cols, questions_sc)))
corrStd_df = corrs.set_index(["id", "wave", "treatment_wave2", "question"]).
↳ join(stds.set_index(["id", "wave", "question", "treatment_wave2"])).
↳ reset_index()

def p_to_stars(p):
```

```

    if p < 0.001: return "***"
    if p < 0.01:  return "**"
    if p < 0.05: return "*"
    return "ns"

g = sns.lmplot(
    corrStd_df,
    x=stdname,
    y=k,
    hue="question",
    height=3.5,
    aspect=1.2,
    scatter_kws={"s": 1, "alpha": 0.15},
    line_kws={"linewidth": 1.3},
    legend=False, # we'll draw a compact one manually
    palette=cmapQuestions,
    hue_order=questions_sc
)

ax = g.axes[0, 0]

slopes, stars_list = [], []
for question in questions_sc:
    subset = corrStd_df[corrStd_df["question"] == question].dropna(
        subset=[stdname, k]
    )
    slope, _, _, p, _ = stats.linregress(
        subset[stdname],
        subset[k]
    )
    slopes.append(slope)
    stars_list.append(p_to_stars(p))
handles = [
    plt.Line2D([0], [0], color=cmapQuestions[q], linewidth=2,
               label=f"{labelMap[q]} = {slope:.2f} {stars}")
    for i, (q, slope, stars) in enumerate(zip(questions_sc, slopes, stars_list))
]
ax.set_ylabel(f"issue weight via {kname}\nusing {'all dots for fitting' if
    ↪fitmode=='fitAllDots' else 'using only voter dots and self for fitting'}")

ax.legend(handles=handles, fontsize=7, frameon=False,
          loc="upper right", handlelength=1.2,
          handletextpad=0.4, labelspace=0.3)

# ax.set_title(f"Issue weight vs. social circle opinion spread", fontsize=9,
    ↪pad=8)

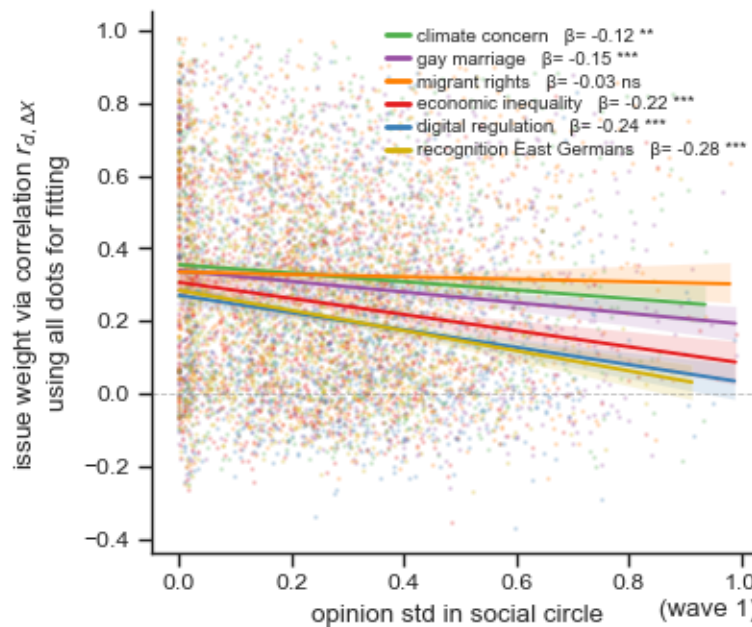
```

```

ax.axhline(0, color="gray", linewidth=0.6, linestyle="--", alpha=0.5) #
    ↳reference line at 0
ax.text(0.99, -0.13, f'(wave {waves[0]})' if len(waves)==1 else '(both waves)',
    ↳va="bottom", ha="right", transform=ax.transAxes)

g.figure.tight_layout()
g.figure.savefig("figs/corrP_stdSC.png", dpi=150, bbox_inches="tight")

```



4 When social circle variance changes for issue q , how does this change the issue weigh?

- Expect: Negative (lens theory)
- Find: negative (both in treatment and control)

```

[7]: # %%
# Relation stdSC to corrP/issue weights
k = "corrP_alpha"
kname = r"r_{d,\Delta X}"
# k = "exp_alpha"
# kname = r"\alpha^{\mathrm{exp}}_{q}"
fig, axes = plt.subplots(2, 3, figsize=(7, 4.5), sharey=True, sharex=True, )

for ax, q in zip(axes.flatten(), questions_sc):

```

```

alpha_change = df_p.pivot_table(index="wave", columns="id",
↳values=f"{k}_{fitmode}_{q}").diff(axis=0).query("wave==2").T.
↳rename(columns={2:"diff"})

std_change = df_p.pivot_table(index="wave", columns="id",
↳values=f"std_socialCircle_ops_{q}").diff(axis=0).query("wave==2").T.
↳rename(columns={2:"std_diff"})

std_change = std_change.join(df_p.query("wave==2")[["id",
↳"treatment_wave2"]].set_index("id"),)
ddd = std_change.join(alpha_change)
for trt, color in cmapTreatment.items():
    subset = ddd[ddd["treatment_wave2"] == trt]

sns.regplot(
    data=subset,
    x="std_diff",
    y="diff",
    ax=ax,
    scatter=False,
    color=color,
    label=None,
    line_kws={"linestyle":"--" if trt==1 else "-"}
)

sns.scatterplot(
    data=ddd,
    x="std_diff",
    y="diff",
    hue="treatment_wave2",
    ax=ax,
    legend=False,
    palette=cmapTreatment,
    alpha=0.2,
    size=1,
)
ax.set_title(f"{q}")
ax.set_xlabel(""); ax.set_ylabel("")
# sns.scatterplot(ddd, y="diff", x="std_diff", hue="treatment_wave2",
↳ax=ax, legend=False, palette=palette, size=2, alpha=0.2, lw=0.1,
↳edgecolor="w")
# sns.scatterplot(ddd, y="diff", x="std_diff", hue="treatment_wave2",
↳ax=ax, legend=False, palette=palette, size=2, alpha=0.2, lw=0.1,
↳edgecolor="w")
# sns.boxplot(alpha_change, x="diff", hue="treatment_wave2", ax=ax2,
↳legend=False, fliersize=False, palette=palette)
# mean_diff = alpha_change.groupby("treatment_wave2")["diff"].mean().
↳reset_index()

```



```

    # sns.stripplot(x="diff", y=None, hue="treatment_wave2", data=mean_diff,
    ↪ax=ax2, legend=False, dodge=True, s=4, marker="s", linewidth=1,
    ↪edgecolor="k", palette=palette)

    slopes, stars_list = [], []
    for trt, group in enumerate(["Control", "Treatment"]):
        subset = ddd[ddd["treatment_wave2"] == trt].dropna(subset=["diff",
    ↪"std_diff"])
        slope, _, _, p, _ = stats.linregress(
            subset["std_diff"],
            subset["diff"]
        )
        slopes.append(slope)
        stars_list.append(p_to_stars(p))

    legend_handles = [
        plt.Line2D([0], [0], color=cmapTreatment[i], linewidth=2,
            label=fr"{group}    $\beta= {slope:.2f}$ {stars}")
        for i, (group, slope, stars) in enumerate(zip(["Control", "Treatment"],
    ↪slopes, stars_list))
    ]

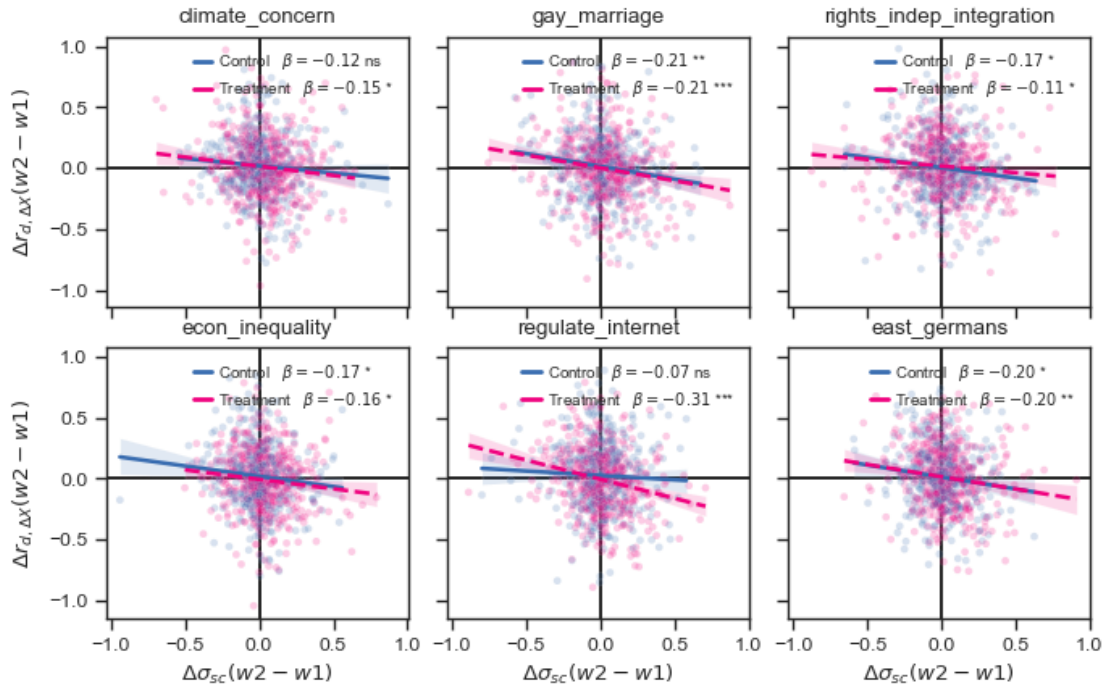
    ax.legend(
        handles=legend_handles,
        # labels=["Control", "Treatment"],
        fontsize=7,
        frameon=False,
        loc="upper right",
        handlelength=1.2,
        handletextpad=0.4,
        labelspacing=0.3,
    )

    for ax in axes[-1,:]:
        ax.set_xlabel(fr"$\Delta \sigma_{\{'sc'\}}\{'(w2-w1)'\}$")
    for ax in axes[:,0]:
        ax.set_ylabel(fr"$\Delta \{kname\}\{'(w2-w1)'\}$")

    for ax in axes.flatten():
        ax.axvline(0,color="k", zorder=-1)
        ax.axhline(0,color="k", zorder=-1)

    fig.tight_layout(h_pad=0)
    plt.savefig(f"figs/corr_varChange_{k}_change.png", dpi=600)

```



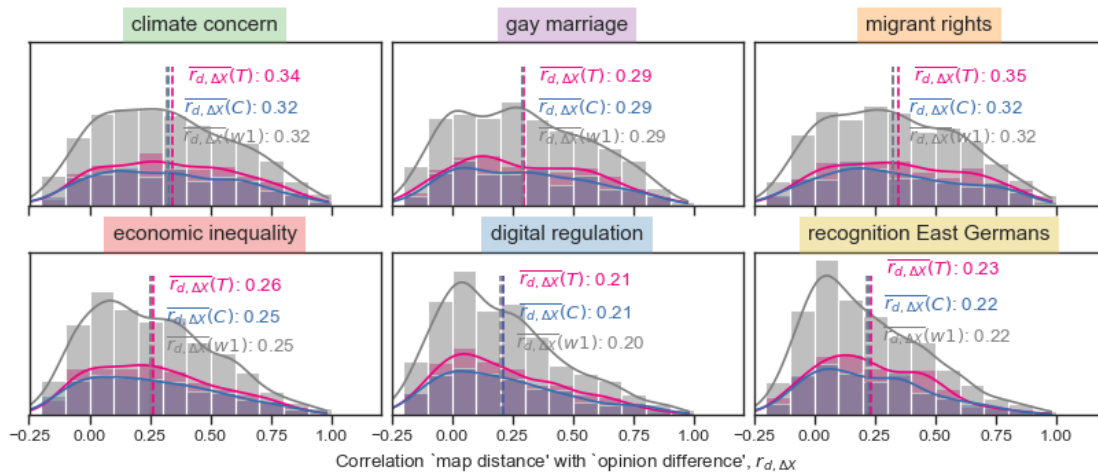
```
[8]: waves = [1,2]
wavecondition = "wave in [1,2]"
corr_cols = [f"corrP_alpha_{fitmode}_{q}" for q in questions_sc]
fig, axes = plt.subplots(2, 3, figsize=(8, 3.5), sharey=True, sharex=True)
df_p["treatment_wave2"] = df_p.treatment_wave2.map({0.0:False, 1.0:True, np.nan:
    ↪ "w1", "w1": "w1"})
for ax, col, q in zip(axes.flatten(), corr_cols, questions_sc):
    sns.histplot(
        df_p.query(wavecondition)[[col, "treatment_wave2"]],
        x = col,
        ax=ax,
        bins=14,
        binrange=(-0.2, 1.2),
        hue="treatment_wave2",
        kde=True,
        stat="density",          # makes KDE and bars scale together properly
        palette=cmapTreatment,#{ "T": "tomato", "C": "k", "w1": "grey"},
        alpha=0.5,
        legend=False,
    )
    mean_T = df_p.query(wavecondition+" and treatment_wave2==True")[col].
    ↪ dropna().mean()
    mean_C = df_p.query(wavecondition+" and treatment_wave2==False")[col].
    ↪ dropna().mean()
```

```

mean_w1 = df_p.query(wavecondition+" and treatment_wave2=='w1'")[col].
↳dropna().mean()

for meanval, y, label in zip([mean_T, mean_C, mean_w1], [0.97,0.75, 0.55],
↳[True, False, "w1"]):
    ax.vlines(
        meanval,
        0,1,
        color=cmapTreatment[label],
        linestyle="--"
    )
    s = r'{r_{d,\Delta X}}'
    ax.text(meanval, ax.get_ylim()[1]*y, rf" $\overline{\text{\textit{s}}}(\text{'w1' if }
↳\text{'w1'}==\text{label else ('T' if label else 'C'))})$:"+f" {meanval:.2f}", ha='left',
↳va='top', color=cmapTreatment[label])
    ax.set_title(labelMap[q], bbox=dict(facecolor=cmapQuestions[q], alpha=0.3,
↳edgecolor='none', pad=4), fontsize=bigfs)
    ax.set_xlim(-0.25, 1.2)
    ax.set_yticks([])
    ax.set_ylabel("")
    ax.set_xlabel("")
    ax.tick_params(axis="x")
    # ax.text( df_p.query(wavecondition+" and treatment_wave2==0")[col].
↳dropna().median(), 2., f" median: {df_p.query(wavecondition+' and
↳treatment_wave2==0')[col].dropna().median():.2f}", ha='left', va='top')
axes[1,1].set_xlabel(fr"Correlation `map distance' with `opinion difference',
↳$\text{\textit{s}}$")
fig.tight_layout(pad=0.6)
plt.savefig("figs/corrP_dist_overTreatment.png")

```



5 Relation of social circle variance with a) liking and b) map distance to voters

- we potentially expected a positive relation for liking the enemy: larger social circle, larger liking of enemy voters
- we expect a negative relation for distance to enemy: larger social circle variance, smaller distance to self

1. Example plots: Greens vs AfD.
2. Heatmap: for all parties

QUESTION: Should we look only at control condition? Wave 1 (but there is no sympathy variable), wave 2 or both?

```
[9]: def slope_and_stars(g, varx, vary):
    res = linregress(g[varx], g[vary])
    slope, p = res.slope, res.pvalue
    if p < 0.001:
        stars = "***"
    elif p < 0.01:
        stars = "**"
    elif p < 0.05:
        stars = "*"
    else:
        stars = ""
    return pd.Series({"slope": slope, "p": p, "stars": stars})

def plot_with_slope(ax, var, condition, party, dot2, title, varname):
    cols = [f"std_socialCircle_ops_{q}" for q in questions_sc] + [var, "dot2",
    ↪ "party"]
    a = df_diff.query(condition).query(f"party=='{party}' and dot2=='{dot2}' and dot1 == 'self'")
    ↪ [cols].dropna()
    a["overall_std"] = a[[f"std_socialCircle_ops_{q}" for q in questions_sc]].
    ↪ mean(axis=1)
    sns.regplot(data=a, x="overall_std", y=var, scatter_kws=dict(s=5, alpha=0.
    ↪ 8), ax=ax)
    ax.set_title(title)
    res = linregress(a["overall_std"], a[var])
    p = res.pvalue
    ax.text(
        0.05, 0.95,
        fr"slope $\beta$ = {res.slope:.3f}" + "\n" + fr"$R^2$ = {res.rvalue**2:.
    ↪ 3f}" + "\n" + fr"$p$ = {p:.3g}" + (( " ***" if p<0.001 else ("**" if p<0.01 else
    ↪ "*")) if p<0.05 else ""),
        transform=ax.transAxes,
        ha="left", va="top",
        fontsize=9,
        bbox=dict(facecolor="white", alpha=0.7, edgecolor="none"),
```

```

)
ax.set_ylabel(varname)
ax.set_xlabel(r"overall social circle SD, $\overline{SD}$")
fig.tight_layout()
return res

```

```

[10]: def plot_slopes_heatmap(var, condition, varname):
    allowedot1 = ['self']
    a = df_diff.query(condition).query(f"dot1 in {allowedot1} and dot2 in {partiesVars} and party in {partiesVars}")[[f"std_socialCircle_ops_{q}" for q in questions_sc]+[var, "dot1", "dot2", "party", "wave"]]
    a["overall_std"] = a[[f"std_socialCircle_ops_{q}" for q in questions_sc]].mean(axis=1)
    results = a.groupby(["dot2", "party"]).apply(slope_and_stars, varx="overall_std", vary=var)
    slope_table = results["slope"].unstack("party").loc[partiesVars, partiesVars]
    stars_table = results["stars"].unstack("party").loc[partiesVars, partiesVars]
    annot_labels = slope_table.round(1).astype(str) + stars_table

    fig, ax = plt.subplots(1, 1)
    s = r'\overline{SD}'
    sns.heatmap(
        slope_table, cmap="coolwarm", vmin=-1, vmax=1,
        cbar_kws={"label": fr"$\beta$: {varname} over ${s}$"},
        ax=ax, annot=annot_labels, fmt="", annot_kws={"fontsize":7}
    )
    ax.set_aspect("equal")
    ax.set_ylabel("evaluated voter of party...")
    ax.set_xlabel("participants who feel closest to party...")
    ax.set_title((
        "(self--voters; " if len(a.dot1.unique())==1 and a.dot1.
unique()=="self" else "(self--voters or voters--voters; ") +
        (f'wave {waves[0]}' if len(a.wave.unique())==1 else 'both waves') +
        (f'; control condition)' if "treatment_wave2==False" in condition else
        ("; treatment condition)" if "treatment_wave2==True" in condition else ';
both conditions'))
        ), x=0, ha="left", fontsize=smallfs)
    fig.autofmt_xdate()

    return results

```

```

[14]: var = "sympathy"
# condition = "wave==2 and treatment_wave2==False"
condition = "wave==2"

```

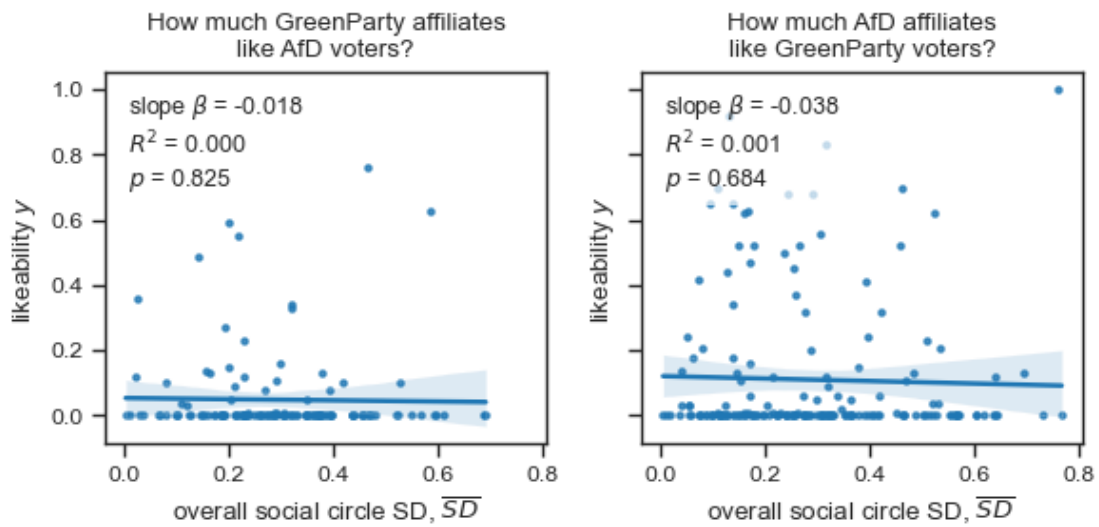
```

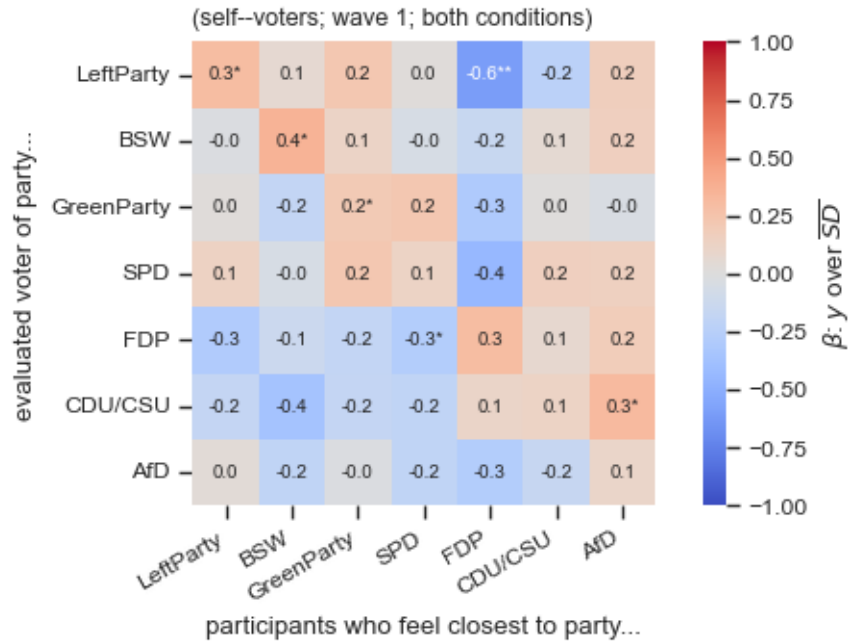
p1 = r'GreenParty'
p2 = r'AfD'

fig, (ax1, ax2) = plt.subplots(1, 2, sharex=True, sharey=True, figsize=(6, 3))
res1 = plot_with_slope(ax1, var, condition, p1, p2, f"How much {p1}\n↳affiliates\nlike {p2} voters?", "likeability $y$")
res2 = plot_with_slope(ax2, var, condition, p2, p1, f"How much {p2}\n↳affiliates\nlike {p1} voters?", "likeability $y$")
plt.savefig(f"figs/correlations_partyXvoter_{var}_{p1}-{p2}.png", dpi=600)

plot_slopes_heatmap(var, condition, varname="$y$")
plt.savefig(f"figs/correlations_partyXvoter_{var}.png", dpi=600)

```

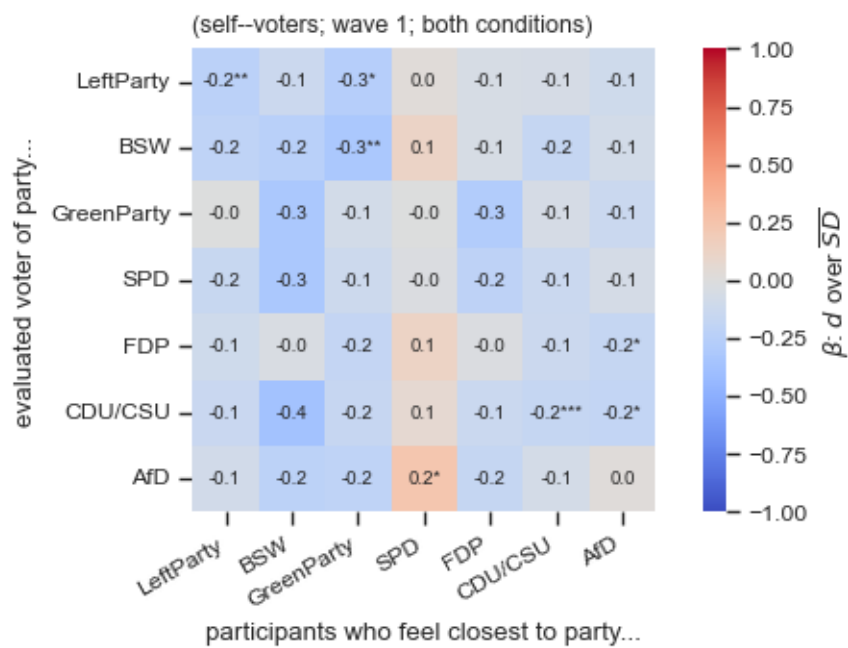
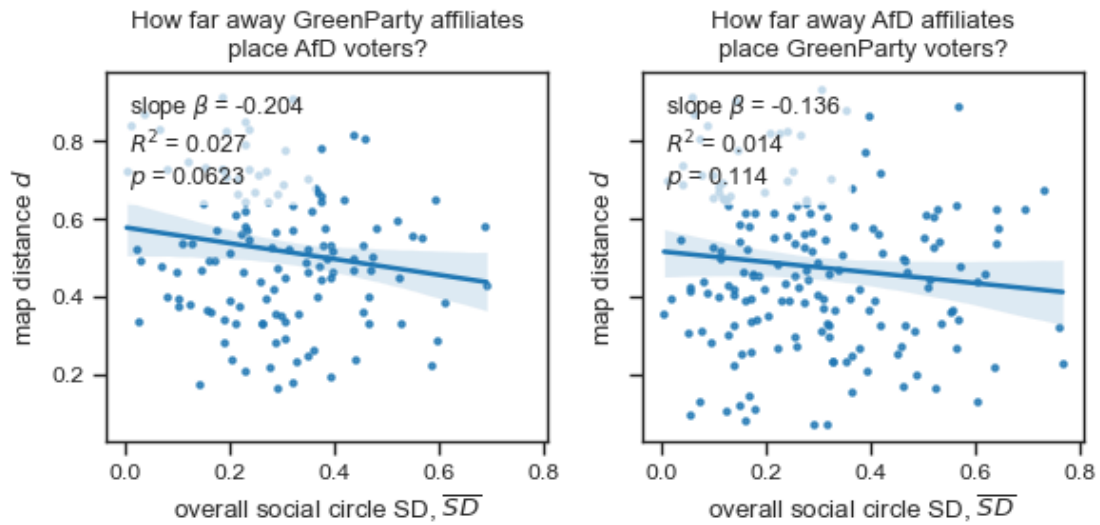




```
[15]: var = "pixel_dist"
#condition = "wave==2 and treatment_wave2==False"
# condition = "wave==2 or wave==1"
condition == "wave==2"
p1 = r'GreenParty'
p2 = r'AfD'

fig, (ax1, ax2) = plt.subplots(1, 2, sharex=True, sharey=True, figsize=(6, 3))
res1 = plot_with_slope(ax1, var, condition, p1, p2, f"How far away {p1}\n↪affiliates\nplace {p2} voters?", "map distance $d$")
res2 = plot_with_slope(ax2, var, condition, p2, p1, f"How far away {p2}\n↪affiliates\nplace {p1} voters?", "map distance $d$")
plt.savefig(f"figs/correlations_partyXvoter_{var}_{p1}-{p2}.png", dpi=600)

plot_slopes_heatmap(var, condition, varname="$d$")
plt.savefig(f"figs/correlations_partyXvoter_{var}.png", dpi=600)
```



[]: